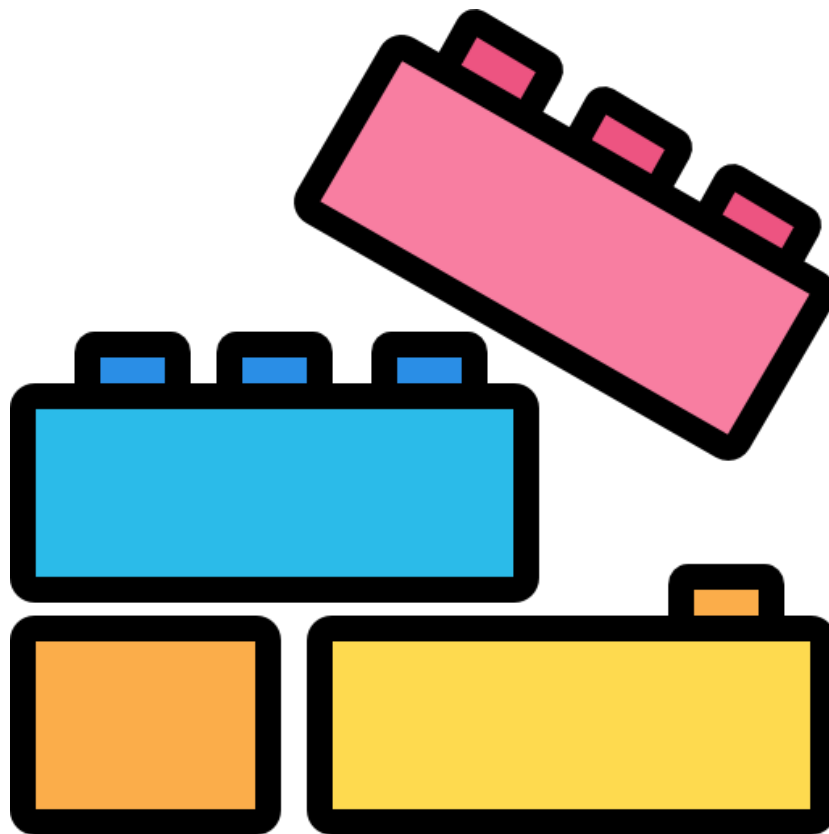


Practical Guide to Kubernetes Deployments



Contact:



Email: vladplan8@gmail.com

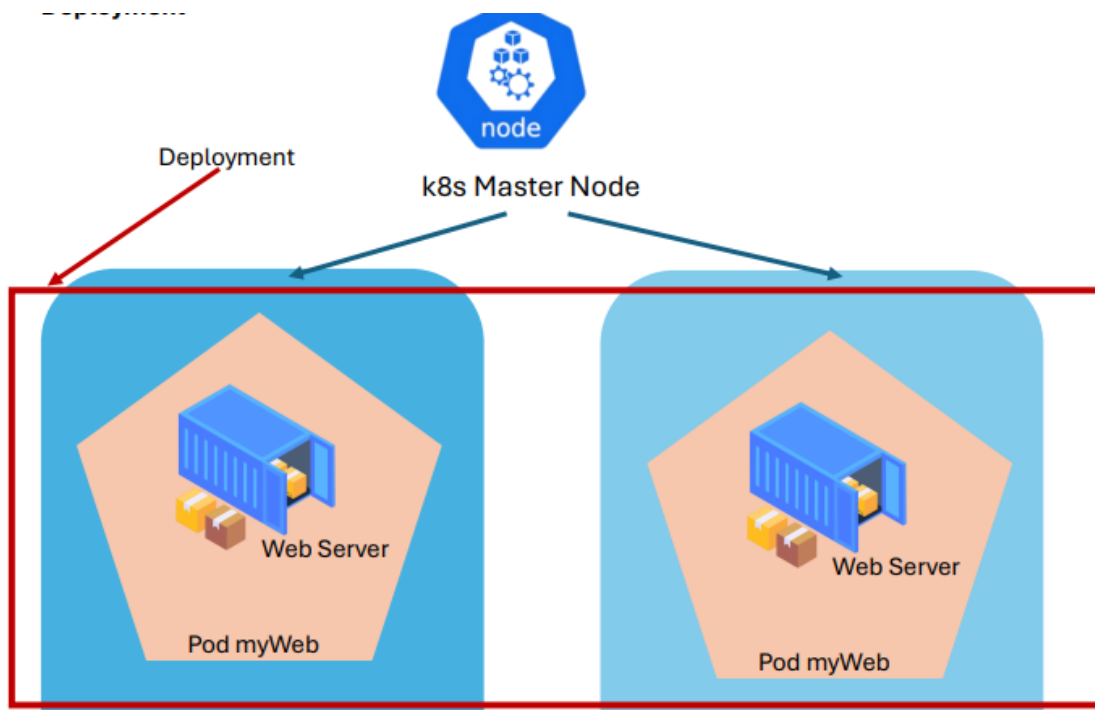


LinkedIn: www.linkedin.com/in/vladyslav-urdenko

Practical Guide to Kubernetes Deployments: From Zero to Autoscaling

In this hands-on lab, we explore the core concepts of Kubernetes Deployments. We start by setting up a local cluster, move to imperative management (CLI commands), and finally transition to declarative management using YAML manifests. We cover scaling, self-healing, rolling updates, and autoscaling.

Today we will do this part:



Setting up the Cluster

First, we need a Kubernetes cluster. For this practice, we use Minikube to create a local cluster with multiple nodes to simulate a real environment.

```
vladyslav@vladyslav-std:~$ minikube start --nodes 2 -p app-cluster
🐶 [app-cluster] minikube v1.38.0 on Ubuntu 24.04 (vbox/amd64)
🌟 Automatically selected the docker driver. Other choices: none, ssh
❗ Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔧 Using Docker driver with root privileges
👍 Starting "app-cluster" primary control-plane node in "app-cluster" cluster
📦 Pulling base image v0.0.49 ...
🔥 Creating docker container (CPUs=2, Memory=3072MB) ...-
```

OR

```
vladyslav@vladyslav-std:~$ minikube start --nodes 2 -p app-cluster --driver=docker
[app-cluster] minikube v1.38.0 on Ubuntu 24.04 (vbox/amd64)
Using the docker driver based on user configuration
```

Explanation:

--nodes 2: We spin up 2 nodes (1 control-plane, 1 worker) to test distribution.

-p app-cluster: We name our profile "app-cluster" to keep it separate from other work.

--driver=docker: We use Docker containers to simulate the nodes.

Verification:

To confirm our nodes are ready:

```
vladyslav@vladyslav-std:~$ kubectl get nodes
NAME                STATUS    ROLES          AGE      VERSION
app-cluster         Ready    control-plane   5m34s    v1.35.0
app-cluster-m02     Ready    <none>          5m1s     v1.35.0
app-cluster-m03     Ready    <none>          92s      v1.35.0
vladyslav@vladyslav-std:~$ kubectl get deploy
No resources found in default namespace.
vladyslav@vladyslav-std:~$
```

Creating Your First Deployment (Imperative)

Instead of creating a single Pod, we create a Deployment. Deployment is a higher-level object that manages Pods, ensuring the desired state is always maintained.

```
vladyslav@vladyslav-std:~$ kubectl create deployment vladyslav-deployment --image vladyslav4hobby/learn_k8s_php
```

Explanation:

This tells Kubernetes to download the specified Docker image and start it inside a Pod. The Deployment controller immediately starts monitoring this, Pod.

Check it:

```
vladyslav@vladyslav-std:~$ kubectl get deploy
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
vladyslav-deployment 0/1      1              0            16s
vladyslav@vladyslav-std:~$
```

Describe Deployment

Like for Pods here we also have describe command to check more info about our item. Try to do it and maybe find container info, ips etc.. It could be useful for your future career, you should know how to get to know more about it.

```
vladyslav@vladyslav-std:~$ kubectl describe pods vladyslav-deployment-76d897db85-cpjmb
```

Scaling the Application

One of Kubernetes' main features is the ability to scale applications up or down instantly to handle load.

```
Normal ScalingReplicaSet 80s deployment-controller Scaled up replica set vladyslav-deployment-76d897db85 from 0 to 1
vladyslav@vladyslav-std:~$ kubectl describe deployment vladyslav-deployment
```

Explanation:

We instructed the controller to ensure 4 copies (replicas) of our application are running. If we check the pods (kubectl get pods), we will see 4 instances starting up.

Understanding ReplicaSets:

When we check the ReplicaSet (kubectl get rs), we see it tracks the "Desired" vs. "Current" state. The ReplicaSet is the specific mechanism the Deployment uses to guarantee the correct number of pods exists.

```
vladyslav@vladyslav-std:~$ kubectl scale deployment vladyslav-deployment --replicas 4
deployment.apps/vladyslav-deployment scaled
vladyslav@vladyslav-std:~$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
vladyslav-deployment-76d897db85-b2ndj 0/1     ContainerCreating   0          9s
vladyslav-deployment-76d897db85-cpjmb 1/1     Running             0          7m13s
vladyslav-deployment-76d897db85-m5qvm 0/1     ContainerCreating   0          9s
vladyslav-deployment-76d897db85-zr5f8 1/1     Running             0          9s
vladyslav@vladyslav-std:~$
```

Kubectl scale deployment (name_of_deployment) --replica (number_of_pods)

Testing Self-Healing

Kubernetes is designed to be resilient. Let's see what happens if a Pod crashes or is deleted.

```
vladyslav@vladyslav-std:~$ kubectl get rs
NAME                                DESIRED    CURRENT    READY    AGE
vladyslav-deployment-76d897db85    4          4          4        8m58s
vladyslav@vladyslav-std:~$
```

If a Pod crashes, is deleted, or fails, the ReplicaSet automatically spins up a new one to replace it. If there are too many Pods running (for example, if a disconnected node rejoins the cluster), it will terminate the extras.

Let's delete our pod

```
vladyslav@vladyslav-std:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
vladyslav-deployment-76d897db85-b2ndj 1/1      Running   0           3m45s
vladyslav-deployment-76d897db85-cpjmb 1/1      Running   0           10m
vladyslav-deployment-76d897db85-m5qvm 1/1      Running   0           3m45s
vladyslav-deployment-76d897db85-zr5f8 1/1      Running   0           3m45s
vladyslav@vladyslav-std:~$ kubectl delete pods vladyslav-deployment-76d897db85-b2ndj
pod "vladyslav-deployment-76d897db85-b2ndj" deleted from default namespace
vladyslav@vladyslav-std:~$
```

And how you can see new will create quickly

```
vladyslav@vladyslav-std:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
vladyslav-deployment-76d897db85-5dzjc 1/1      Running   0           43s
vladyslav-deployment-76d897db85-cpjmb 1/1      Running   0           12m
vladyslav-deployment-76d897db85-m5qvm 1/1      Running   0           5m3s
vladyslav-deployment-76d897db85-zr5f8 1/1      Running   0           5m3s
vladyslav@vladyslav-std:~$
```

Horizontal Pod Autoscaling (HPA)

Manual scaling is useful, but Autoscaling allows the cluster to react to traffic spikes automatically.

```
vladyslav@vladyslav-std:~$ kubectl autoscale deployment vladyslav-deployment --min=4 --max=6 --cpu=80%
```

Explanation:

Min/Max: We set a lower limit of 4 pods and an upper limit of 6.

CPU Threshold: If the average CPU usage across pods exceeds 80%, Kubernetes will add more pods (up to the max) to handle the load.

Check it:

```
vladyslav@vladyslav-std:~$ kubectl get hpa
NAME                                REFERENCE                                TARGETS                                MINP
ods    maxpods    replicas    age
vladyslav-deployment    Deployment/vladyslav-deployment    cpu: <unknown>/80%    4
6            4            2m55s
```

Rolling Updates & History

Deployments allow us to update our application without downtime. We can change the underlying image (updating to a new version) seamlessly.

```
vladyslav@vladyslav-std:~$ kubectl set image deployment/vladyslav-deployment learn
-k8s-php-qn67p=grafana/grafana:latest --record
Flag --record has been deprecated, --record will be removed in the future
```

Explanation:

This updates the container image from our PHP app to Grafana. Kubernetes performs a Rolling Update: it spins up new pods with the new image and shuts down old ones one by one, ensuring service availability.

Checking History:

```
vladyslav@vladyslav-std:~$ kubectl rollout history deployment/vladyslav-deployment
deployment.apps/vladyslav-deployment
REVISION    CHANGE-CAUSE
1            <none>
```

`kubectl rollout history deployment/(name_of_deployment)`

This shows the "revisions" of our deployment. If the new update has a bug, we can instantly rollback:

```
vladyslav@vladyslav-std:~$ kubectl rollout undo deployment/vladyslav-deployment
```

But if I want to specify my history move, maybe I want to turn back from step 3 to 1.

Also can do this..

```
vladyslav@vladyslav-std:~$ kubectl rollout history deployment/vladyslav-deployment
deployment.apps/vladyslav-deployment
REVISION  CHANGE-CAUSE
3         <none>
4         kubectl set image deployment/vladyslav-deployment learn-k8s-php-qn67p=grafana/grafana:latest --record=true
```

Use --to-revision=

```
vladyslav@vladyslav-std:~$ kubectl rollout undo deployment/vladyslav-deployment --to-revision=3
deployment.apps/vladyslav-deployment rolled back
vladyslav@vladyslav-std:~$
```

Hmm.. And If I want just quickly renew to latest version, like restart, when I dont need to change container so much.. Look..

```
vladyslav@vladyslav-std:~$ kubectl rollout restart deployment/vladyslav-deployment
deployment.apps/vladyslav-deployment restarted
vladyslav@vladyslav-std:~$
```

It will deploy this version again, renew

Declarative Management (YAML Manifests)

While CLI commands are good for quick tests, YAML files are the standard for production (Infrastructure as Code).

Create folder and file for work

```
vladyslav@vladyslav-std:~$ mkdir Lesson_Depl
vladyslav@vladyslav-std:~$ cd Lesson_Depl/
vladyslav@vladyslav-std:~/Lesson_Depl$ ls
vladyslav@vladyslav-std:~/Lesson_Depl$ touch simple-1-deploy.yaml
vladyslav@vladyslav-std:~/Lesson_Depl$ ls
simple-1-deploy.yaml
```

Minimal set!! MIN!

```
apiVersion : apps/v1
kind: Deployment
metadata:
  name: my-web-deployment
  labels:
    app : grafana-application
    owner : Vladyslav
spec:
  selector:
    matchLabels:
      project: aamm
  template:
    metadata:
      labels:
        project: aamm
    spec:
      containers:
        - name : grafana-web
          image : grafana/grafana:latest
          ports:
            - containerPort: 3000
```

simple-1-deploy.yaml [+]

21,34

-- SYÖTTÖ --

Breakdown

Key / Section	Explanation
apiVersion	Specifies the version of the Kubernetes API used to create this object (apps/v1 is standard for deployments).
kind	Defines the type of resource. A Deployment ensures a specific number of pod copies are running.
metadata	Contains data that helps uniquely identify the object, such as its name (my-web-deployment).
labels	Key-value pairs used to organize and group resources (e.g., owner: Vladyslav).
spec	The "desired state." This tells Kubernetes exactly what you want the application to look like.

selector	Tells the Deployment which Pods it is responsible for managing by looking for matching labels.
matchLabels	The specific criteria for the selector. It looks for Pods labeled project: aamm.
template	The blueprint for the Pods that will be created. It contains its own metadata and spec.
containers	A list of the actual software containers to run inside the Pod.
image	The specific Docker image to download and run (grafana/grafana:latest).
containerPort	The network port the application inside the container is listening on (3000).

We created simple-1-deploy.yaml to define a basic Deployment running Grafana.

Command:

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl apply -f simple-1-deploy.yaml
deployment.apps/my-web-deployment created
vladyslav@vladyslav-std:~/Lesson_Depl$
```

And now it works:

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
my-web-deployment   1/1     1            1           66s
vladyslav-deployment 4/4     4            4           3h11m
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Deployment with Replicas

We modified the file to **simple-2-deploy.yaml**, setting replicas: 3.

Explanation:

By changing the file and re-running kubectl apply, we simply declare "I want 3 replicas now." Kubernetes calculates the difference and adds the necessary pods.

I just copy my file and edit this:

```
vladyslav@vladyslav-std:~/Lesson_Depl$ cp simple-1-deploy.yaml simple-2-deploy.yaml
vladyslav@vladyslav-std:~/Lesson_Depl$ ls
simple-1-deploy.yaml  simple-2-deploy.yaml
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Write to the file:

```
apiVersion : apps/v1
kind: Deployment
metadata:
  name: my-graf-deployment-replic
  labels:
    app : grafana-application
    env: test
    owner : Vladyslav
spec:
  replicas: 3
  selector:
    matchLabels:
      project: monitoring
  template:
    metadata:
      labels:
        project: monitoring
    spec:
      containers:
        - name : monitoring-web
          image : grafana/grafana:latest
          ports:
            - containerPort: 3000
```

Important I highlighted in the red color.

Check our work:

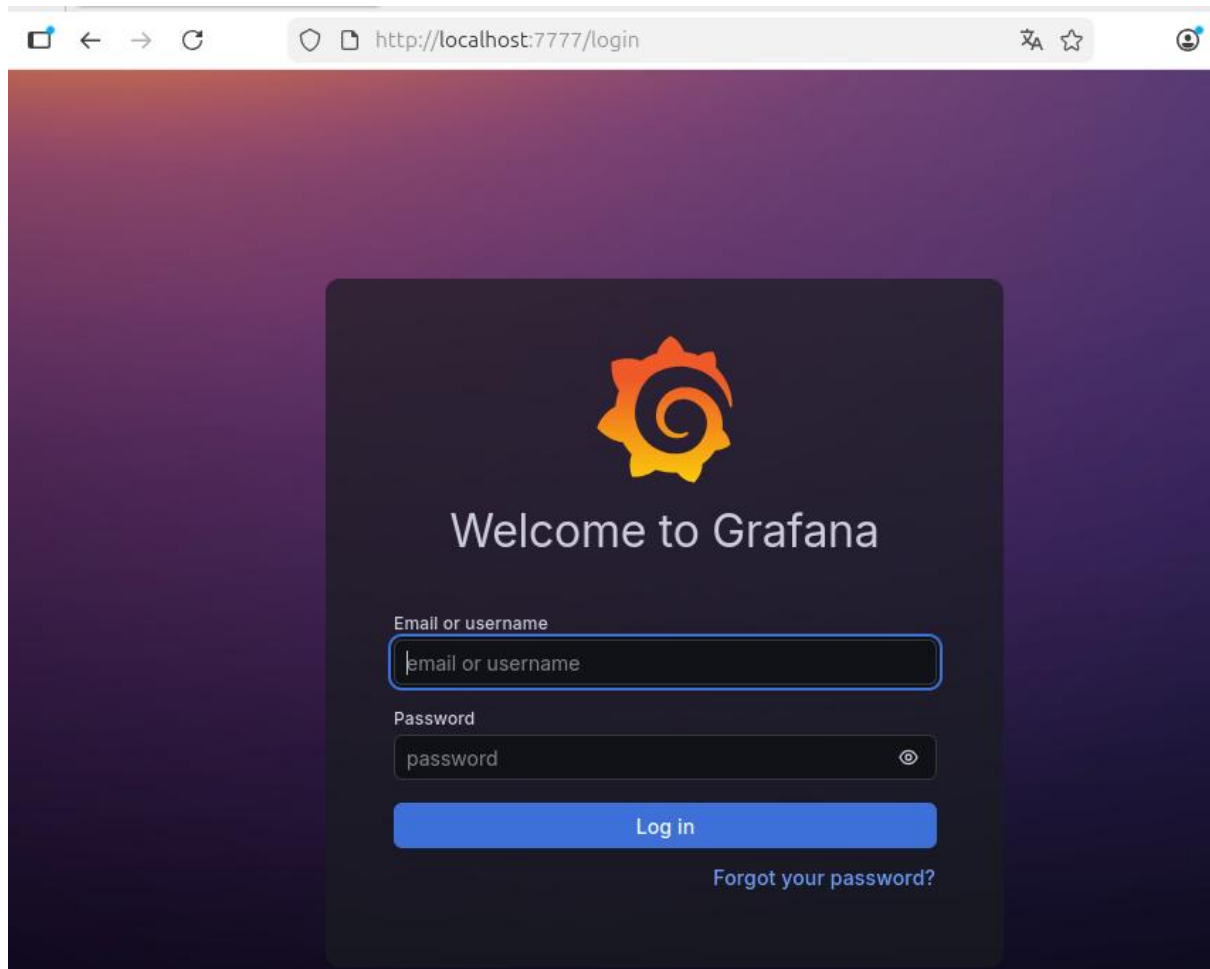
```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl apply -f simple-2-deploy.yaml
deployment.apps/my-graf-deployment-replic created
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl get deploy
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
my-graf-deployment-replic          3/3      3              3            17s
my-web-deployment                  1/1      1              1            8m28s
vladyslav-deployment                4/4      4              4            3h18m
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Accessing the Application

Since these pods are running inside the cluster (Minikube), we cannot access them directly from our browser without exposing them. We used Port Forwarding for testing.

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl port-forward my-graf-deployment-rep  
lic-6db44c45d6-nfldj 7777:3000  
Forwarding from 127.0.0.1:7777 -> 3000  
Forwarding from [::1]:7777 -> 3000
```

Open browser: localhost:7777



Yeps, everything is fine :0

Explanation:

This creates a tunnel from our local machine's port 7777 to the container's port 3000.

Result: We can now visit <http://localhost:7777> in our browser and see the Grafana login screen.

Autoscaling Manifest

I also created file for third project...

```
vladyslav@vladyslav-std:~/Lesson_Depl$ ls -ll
yhteensä 12
-rw-rw-r-- 1 vladyslav vladyslav 404 helmi  19 16:30 simple-1-deploy.yaml
-rw-rw-r-- 1 vladyslav vladyslav 455 helmi  19 16:40 simple-2-deploy.yaml
-rw-rw-r-- 1 vladyslav vladyslav 455 helmi  19 16:56 simple-3-deploy.yaml
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Some changes here:

```
apiVersion : apps/v1
kind: Deployment
metadata:
  name: my-graf-deployment-autoscaling
  labels:
    app : grafana-application
    env: test
    owner : Vladyslav
spec:
  replicas: 2
  selector:
    matchLabels:
      project: monitoring-scale
  template:
    metadata:
      labels:
        project: monitoring-scale
    spec:
      containers:
        - name : monitoring-scaling-web
          image : grafana/grafana:latest
          ports:
            - containerPort: 3000
```

And our autoscaling configuration

--- (is a common thing for YAML files)

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-autoscaling
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-graf-deployment-autoscaling
  minReplicas: 3
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 70
~
~
```

HELP for you!!

Key / Section	Explanation
kind	Defined as HorizontalPodAutoscaler. This resource automatically scales the number of Pods in a replication controller, deployment, or replica set.
scaleTargetRef	The "target" is this autoscaler watches. It points specifically to your Deployment named my-graf-deployment-autoscaling.
minReplicas	The "floor." No matter how low the traffic gets, Kubernetes ensures at least 3 Pods are always running.
maxReplicas	The "ceiling." No matter how high the traffic gets, it will never create more than 5 Pods (to control costs/resources).
metrics	The specific rules are used to calculate when to scale.

resource	Indicates scaling is based on hardware usage rather than custom external metrics (like packet count).
averageUtilization	The trigger point. • CPU: If average usage across pods exceeds 60%, add more pods. • Memory: If average usage exceeds 70%, add more pods.

Look for indentation!!

Let's create our deployment after all and check it:

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl apply -f simple-3-deploy.yaml
deployment.apps/my-graf-deployment-autoscaling created
horizontalpodautoscaler.autoscaling/my-autoscaling created
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Now you see that it created two things

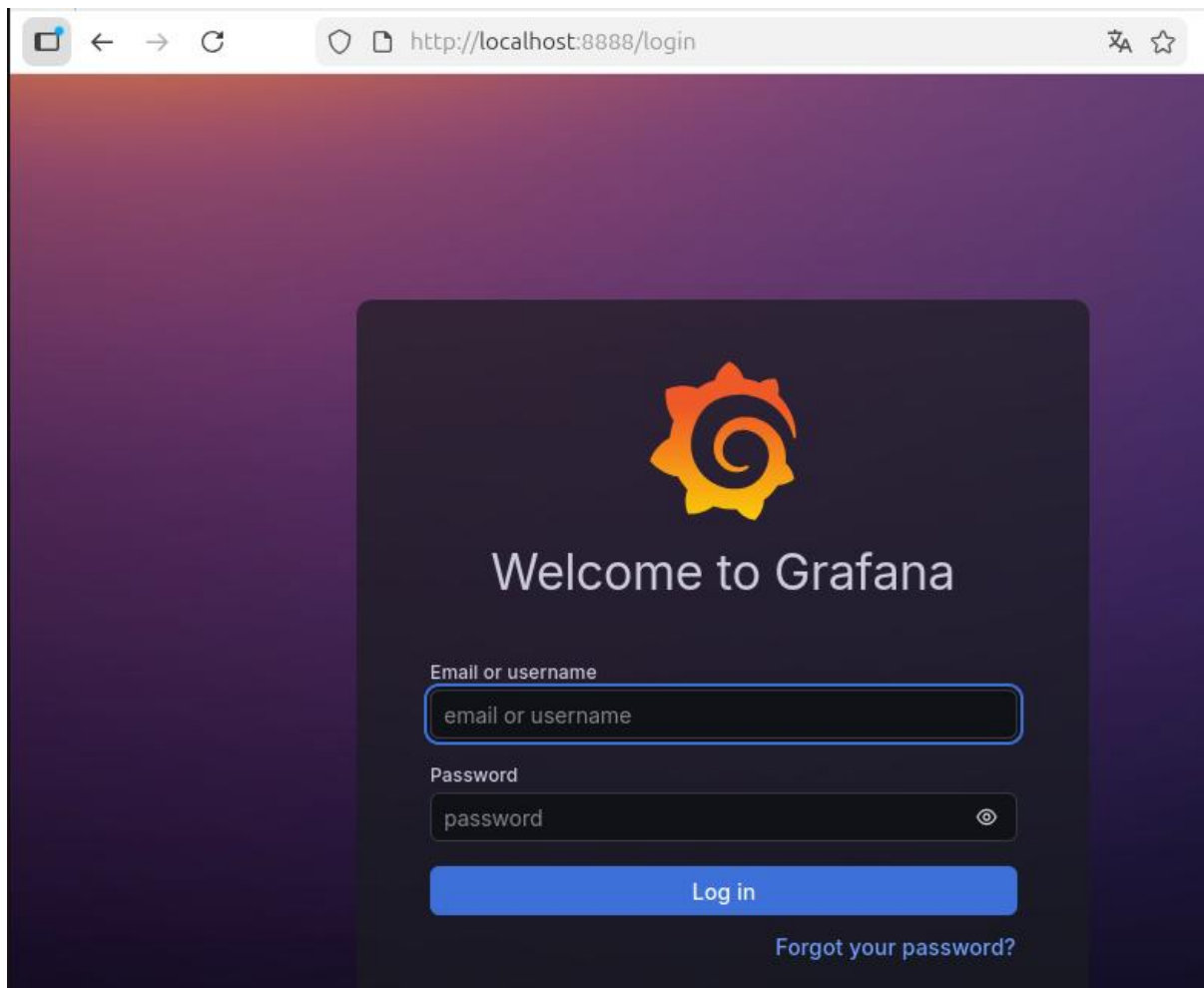
And check our pods, now there are a lot of them

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
my-graf-deployment-autoscaling-5b5c7fd8f-4f8x9   1/1     Running   0           48s
my-graf-deployment-autoscaling-5b5c7fd8f-hv2rz   1/1     Running   0           48s
my-graf-deployment-autoscaling-5b5c7fd8f-w9ftx   1/1     Running   0           32s
my-graf-deployment-replic-6db44c45d6-nfldj       1/1     Running   0           41m
my-graf-deployment-replic-6db44c45d6-rnzpq       1/1     Running   0           41m
my-graf-deployment-replic-6db44c45d6-wh9sk       1/1     Running   0           41m
my-web-deployment-66447bd5b4-jbwqb               1/1     Running   0           49m
vladyslav-deployment-7d9b97bd9d-725td           1/1     Running   0           3h11m
vladyslav-deployment-7d9b97bd9d-cb784           1/1     Running   0           3h11m
vladyslav-deployment-7d9b97bd9d-h22lq           1/1     Running   0           3h11m
vladyslav-deployment-7d9b97bd9d-lslpk           1/1     Running   0           3h11m
vladyslav@vladyslav-std:~/Lesson_Depl$
```

Also check with port-forwarding

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl port-forward my-graf-deployment-autoscaling-5b5c7fd8f-4f8x9 8888:3000
Forwarding from 127.0.0.1:8888 -> 3000
Forwarding from [::1]:8888 -> 3000
Handling connection for 8888
Handling connection for 8888
Handling connection for 8888
Handling connection for 8888
```

Open browser, now we listen port 8888..



Cleanup

To save resources, we clean up the environment after the lab.

```
vladyslav@vladyslav-std:~/Lesson_Depl$ kubectl delete -f simple-1-deploy.yaml
deployment.apps "my-web-deployment" deleted from default namespace
vladyslav@vladyslav-std:~/Lesson_Depl$
```

And..

```
vladyslav@vladyslav-std:~$ kubectl delete nodes --all
node "app-cluster" deleted
node "app-cluster-m02" deleted
node "app-cluster-m03" deleted
vladyslav@vladyslav-std:~$
```

Cheatsheet: Common Deployment Commands

Command	Description
kubectl create deployment <name> --image=	Create a new Deployment
kubectl scale deployment <name> --replicas=<n>	Scale manually to n pods
kubectl autoscale deployment <name> ...	Set up automatic scaling based on CPU
kubectl set image deployment/<name> <container>=	Update the application image
kubectl rollout undo deployment/<name>	Rollback to the previous version
kubectl port-forward <resource> <local>:<remote>	Access app locally for testing